Connecting Claude to Power BI with the Model Context Protocol

I've been watching the Model Context Protocol (MCP) space closely since Anthropic open-sourced it. The idea is simple but powerful: give LLMs a standardized way to connect to external data sources and tools. I'd already played around with the Snowflake MCP server through Claude Desktop, and the experience of just *asking questions* about your data in plain English was compelling. But I kept coming back to the same thought — what about Power BI?

Power BI is where most of the business data I work with actually lives. Microsoft has released an MCP server for Power BI, and I wanted to build something that would let me have a genuine conversation with my Power BI datasets. Not just a one-shot query, but a back-and-forth where Claude could call tools, inspect results, decide it needs more information, and keep going until it has a proper answer.

So I built it.

[INSERT HERO IMAGE: Screenshot of the Streamlit chat UI with a conversation showing a natural language question and a rendered chart response HERE]

What This Thing Actually Does

At its core, this is a chat interface — available as both a CLI and a browser-based Streamlit app — where you type a question in plain English and Claude figures out what Power BI data it needs, fetches it via MCP, and gives you an answer. Often with a chart.

Ask something like "*What were our top 10 products by revenue last quarter?*" and Claude will discover the available Power BI tools, execute the right DAX query, get the results back, and render a bar chart — all without you writing a single line of DAX.

The key word there is "discovers." Claude doesn't have a hardcoded list of queries it can run. At startup, the app connects to the Power BI MCP server, asks it what tools are available, and converts those into Claude's tool format. Claude then decides which tools to call based on your question. It's genuinely agentic.

[INSERT ARCHITECTURAL DIAGRAM: High-level flow showing User → Streamlit/CLI → Agent Orchestrator → Claude API + MCP Client → Power BI MCP Server → Power BI Dataset HERE]

The Agentic Loop

This is the part I find most interesting. The orchestrator doesn't just send your question to Claude and return the response. It runs a loop:

1. Send your message (plus conversation history) to Claude, along with the available Power BI tools
2. Claude responds — either with a final answer, or with a request to call one or more tools
3. If Claude wants to call tools, execute them against Power BI via MCP
4. Feed the tool results back to Claude
5. Repeat until Claude decides it has enough information to give you a final answer

This means Claude can chain multiple queries together. It might look at the available datasets first, then query a specific table, then ask a follow-up to clarify an anomaly it spotted in the data. The loop is bounded (max 10 iterations) so it won't run forever, but in practice most questions resolve in one or two tool calls.

```
for iteration in range(max_iterations):
    response = await claude_client.send_message(messages, tools)

    if response.stop_reason == "end_turn":
        break  # Claude has its answer
    elif response.stop_reason == "tool_use":
        tool_results = await execute_tools(response.tool_calls)
        messages.extend(tool_results)
        # Loop continues — Claude sees the results and decides next steps
```

The MCP Connection Challenge

Here's where things got interesting. Most MCP servers use Server-Sent Events (SSE) for communication — that's the standard transport in the MCP spec. The Power BI MCP server uses HTTP POST with JSON-RPC instead. So I couldn't just use the standard MCP SDK client out of the box.

I ended up building a custom `SimpleMCPClient` with its own `HTTPTransport` layer. It handles the JSON-RPC request/response pattern, manages request IDs, and injects the Azure authentication token into every call. It's not a lot of code, but it was the kind of thing where you spend an afternoon wondering why nothing works before you realize the transport layer is just... different.

[INSERT DIAGRAM: MCP communication flow showing JSON-RPC request/response over HTTP POST, with auth token injection HERE]

Authentication

Speaking of auth — the app uses Microsoft's MSAL library with the device code flow. On first run, you get a URL and a code to punch into your browser. After that, the token gets cached locally and auto-refreshes. It's the same flow you'd use for any Azure-authenticated CLI tool, and it means you don't need to store passwords or manage service principals for development use.

The Streamlit UI

I wanted a proper chat experience, not just a terminal. Streamlit was the obvious choice for a quick, interactive UI, but it introduced its own set of challenges.

The big one: Streamlit reruns your entire script on every interaction. Every button click, every chat message — the whole script executes top to bottom. That's fine for simple apps, but when you have persistent async connections to external services, it's a problem. An `httpx.AsyncClient` created in one script run can't be used in the next because the event loop is gone.

[INSERT SCREENSHOT: The Streamlit sidebar showing connection status, available tools, and settings HERE]

The solution was an `AsyncRunner` class that maintains a single persistent daemon thread with its own event loop. All async operations — MCP calls, Claude API requests — get submitted to this one loop via `asyncio.run_coroutine_threadsafe`. The loop survives across Streamlit reruns, so connections stay alive.

I also had to manually manage the MCP connection's context manager. Normally you'd use `async with client.connect()` and let Python handle cleanup. But Streamlit's rerun behavior would trigger the cleanup and kill the connection. So I enter the context manager manually on connect and just... never exit it. It's not elegant, but it works.

Automatic Chart Detection

One of the things that makes the experience feel polished is automatic chart rendering. When Claude returns tool results that contain tabular data, the app analyzes the data and decides whether a chart would be useful — and if so, what type.

The logic is straightforward: if there's one row, show a metric card. If there's a datetime column, use a line chart. If percentages sum to roughly 100, use a pie chart. Otherwise, default to a bar chart. The user can also hint at what they want — include "pie chart" in your question and it'll override the auto-detection.

The charts are rendered with Altair, which gives you interactive Vega-Lite visualizations that look good out of the box.

[INSERT SCREENSHOT: Example of auto-detected chart — a bar chart showing product revenue data rendered in the Streamlit UI HERE]

The Stack

For anyone who wants to dig in or build something similar:

- **LLM**: Claude via the Anthropic API (with full tool use support)
- **Protocol**: Model Context Protocol over custom HTTP POST transport
- **Auth**: MSAL device code flow with local token caching
- **Web UI**: Streamlit with a persistent async event loop
- **CLI**: prompt-toolkit REPL with Rich formatting
- **Charts**: Altair (declarative, Vega-Lite based)
- **Config**: Pydantic settings with `.env` file support

What I Learned

The MCP ecosystem is still young, and the implementations vary. Don't assume every MCP server follows the spec to the letter — the Power BI server's HTTP POST transport caught me off guard. Building your own transport layer isn't hard, but you need to know that you need to.

Streamlit and async Python are not natural friends. If you're building anything with persistent connections in Streamlit, plan for the rerun behavior from the start. The persistent event loop pattern I landed on works well, but it took a few rounds of "why is my connection dead?" to get there.

And the agentic loop pattern — letting Claude decide when it has enough information — is more powerful than I expected. Most questions are simple enough that one tool call handles it. But occasionally Claude will chain together three or four calls, exploring the data in a way that feels genuinely intelligent. That's when this approach really shines compared to a traditional dashboard.

[INSERT SCREENSHOT: Example of a multi-turn conversation where Claude chains multiple tool calls to answer a complex question HERE]

Wrapping Up

This started as a weekend experiment to see if I could get Claude talking to Power BI through MCP. It turned into something I actually use day-to-day for quick data exploration. The combination of natural language, an agentic loop, and automatic visualization means I can go from question to insight without context-switching into Power BI Desktop or writing DAX.

The code is on GitHub — feel free to take a look, fork it, or tell me what I'm doing wrong.